Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Slides include material from Introduction to Algorithms, 4th Edition, MIT Press by Cormen, Leiserson, Rivest, and Stein.

Lecture 8

- Recommended reading:
  - Chap. 6 (Heaps, Heapsort, Priority Queues)
  - Probability Handout on EdStem
  - Appendix C (Counting and Probability)
- Reading ahead
  - Chap. 5 Randomized Algorithms
  - Chap. 7 Quicksort

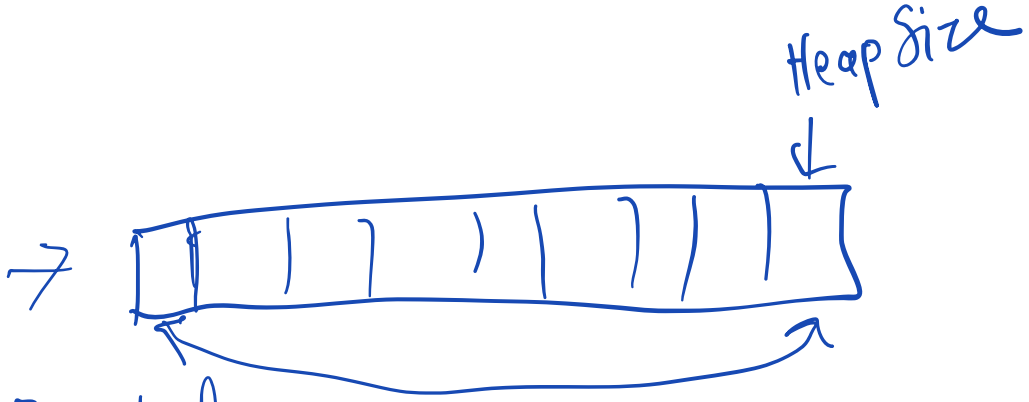
Worst-case Runtime of Heapsort (Section A only)

- Worst-Case Runtime of Heapsort?
  - Build-Max-Heap takes time $\theta(n)$
  - $n - 1$ loop iterations
    - Each one

      exchanges A[1] and A[i] $\theta(1)$ runtime

      decrements A.heapsize $\theta(1)$ runtime

      runs Max-Heapify(A,i) $O(\log n)$ runtime
    - So each loop iteration takes time $O(\log n)$
  - $\Rightarrow$ Worst-case runtime of Heapsort is $O(n \log n)$

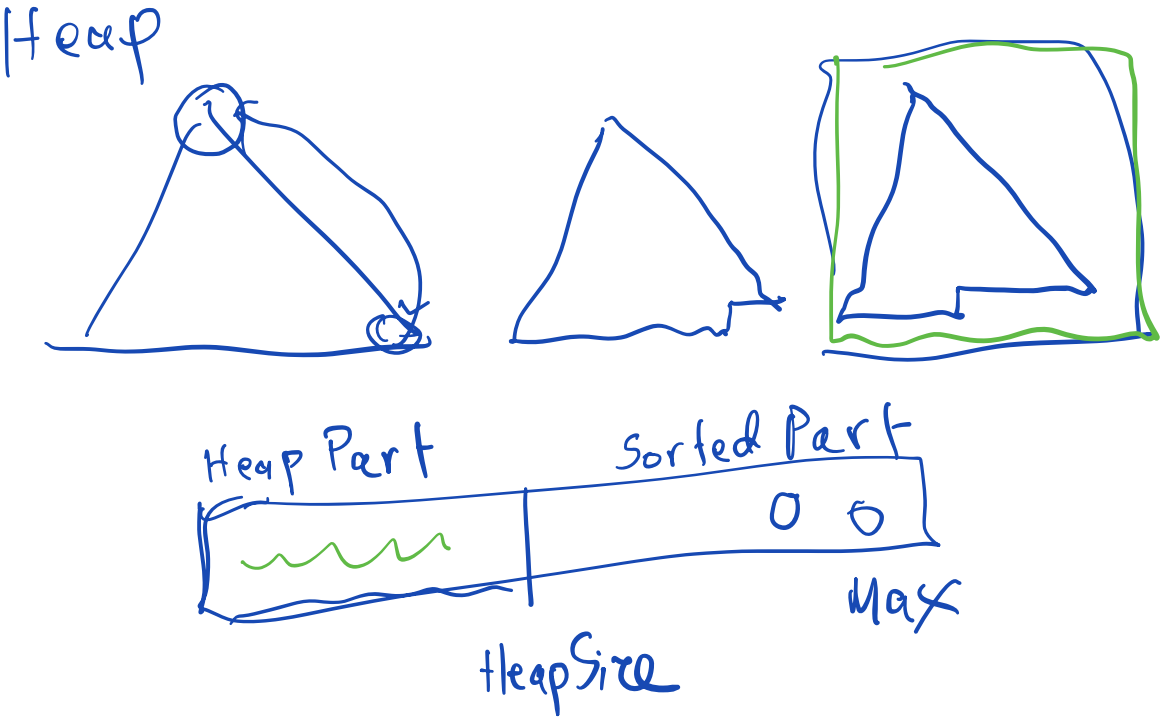


Build Heap

~~Repeat~~ for i=n down to 2

Exchange A[1] with A[A.heapSize]

A.Heapsize --


Heap
Heap Part
Sorted Part
0 0
Max
HeapSize



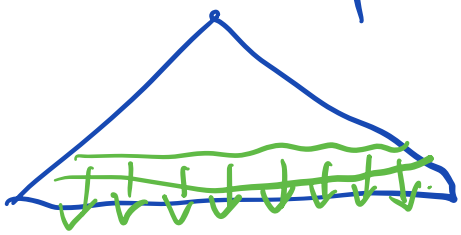
Shrink size of heap by $\frac{1}{2}$

First $\frac{n}{2}$ heapify performed in loop of heap sort in worst case perform $\Omega(\log n)$ comparisons.

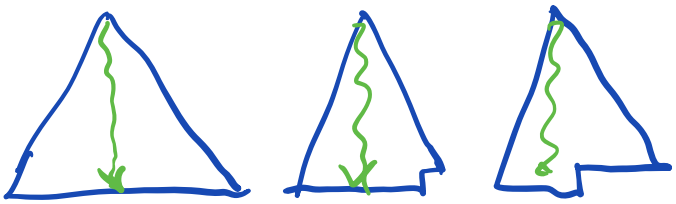
- Worst-case runtime of Heapsort is $\Omega(n \log n)$ too!
- First $n/2$ calls to Max-Heapify are on heaps with $\geq n/2$ elements
  and those heaps each have size $\geq \log_2(n/2) = \log_2 n - 1$
  (ignoring floors and ceilings)
- $\Rightarrow$ First $n/2$ calls to Max-Heapify **each** take time $\geq c(\log_2 n - 1)$ in worst case (worst-case: element at root has to bubble all the way down to a leaf)
- $\Rightarrow$ Worst-case runtime of Heapsort is at least $(n/2) * c(\log_2 n - 1)$
- $\Rightarrow$ Worst-case runtime of Heapsort is $\Omega(n \log n)$

- So worst-case runtime of Heapsort is $\theta(n \log n)$

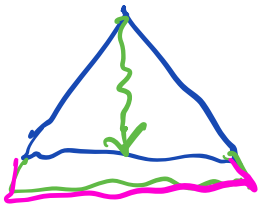
Build Heap



Heap sort



$n/2$ heapifies



Priority Queue

Priority Queue is an Abstract Data Type

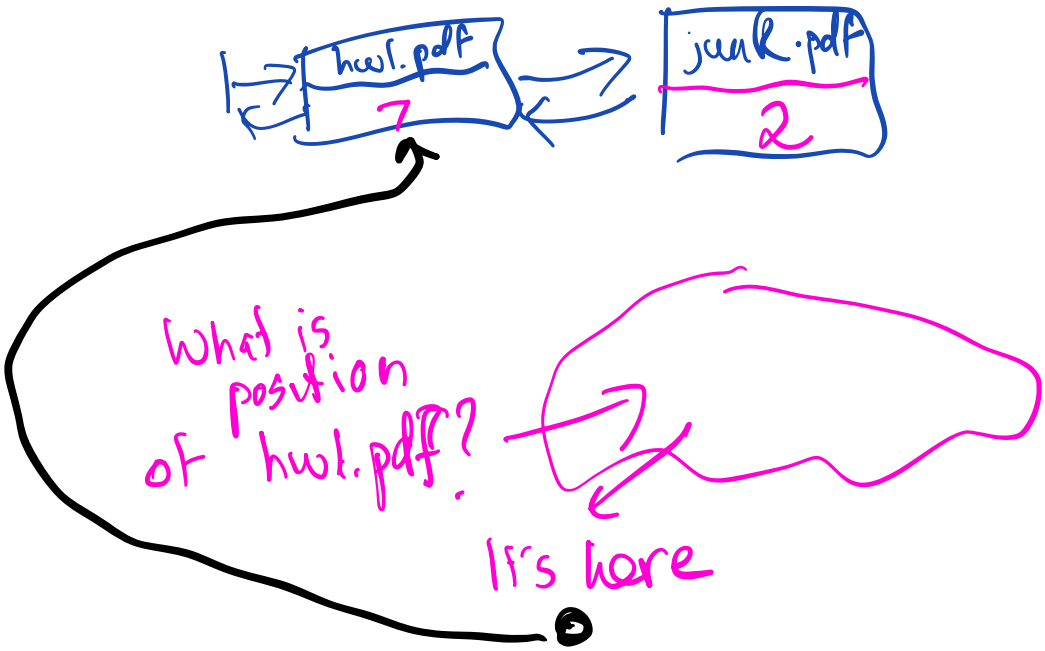
- Stores set S of elements
- (Max) Priority queue implementation must support the following operations
  - Insert(S,x,k) Insert element x into S with key k
  - Maximum(S) Return element in S with largest key (but don't remove)
  - Extract-Max(S) Remove and return element in S with largest key
  - Increase-Key(S,x,k) Increase key of element x of S to k (assume it is less than k)

- Could implement priority queue (PQ) using different data structures
- Example: Implementation of PQ using unsorted doubly-linked list
  - Insert(S,x,k) Insert element x into S with key k
    - How? Insert element x with key k at front of doubly-linked list
    - Worst-case Runtime of Insert(S,x,k)? $\theta(1)$
  - Maximum(S) Return element in S with largest key (but don't remove)
    - How? Linear search (since doubly-linked list is unordered)
    - Worst-case Runtime? $\theta(n)$ where n is number of elements in the priority queue
  - Extract-Max(S) Remove and return element in S with largest key
    - How? Look at all elements to determine max, then delete it from doubly linked list
    - Worst-case Runtime? $\theta(n)$ to determine max, $\theta(1)$ to delete
  - Increase-Key(S,x,k) Increase key of element x of S to k (assume it is less than k)
    - How? Linear search to find x, then increase key of x
    - Worst-case Runtime? $\theta(n)$ to find x, $\theta(1)$ to increase key of x

IncreaseKey and "Handle"

- In order to do IncreaseKey(A,x,k), need to be able to locate element x in the data structure you're using to store the elements of the priority queue
- Implement a "handle" for each element x that maps between elements x and their locations in the data structure
  - e.g., if data structure is a doubly-linked list
    - map element x to pointer that points to (the list node storing) x in the doubly-linked list
  - (also map backwards, from location of x in the data structure to x)

- Handle must be kept updated during PQ operations.
- One popular implementation of handle
  - Use hash-table to map elements to ~~array~~ positions in data structure storing the PQ.
- This implementation has $O(1)$ overhead "on average" ~~per position change~~ per mapping of an elt x to its position in ~~and map.~~ & but worst case is $O(n)$


hwk.pdf
7
junk.pdf
2
What is position of hwk.pdf?
It's here

- Going forward, we'll exclude the runtime of mapping operations from the runtimes
  - e.g., For the *doubly-linked list implementation* of the priority queue, we'll say that the worst-case runtime of Increase-Key(S,x,k) is $\theta(1)$

    (excluding the time taken for the mapping operation of locating x in the doubly-linked list)

Heap implementation of PQ

- Usually better to implement PQ using a **heap**
  - but need to define heap procedures to support the PQ operations
    - Insert(S,x,k)
    - Maximum(S)
    - Extract-Max(S)
    - Increase-Key(S,x,k)
  - For heap sort we only defined
    - Build-Max-Heap(A)
    - Max-Heapify(A,i)
  - We'll now define heap implementations of the above PQ operations, called e.g., Max-Heap-Maximum, Max-Heap-Extract-Max, etc.

- A[i].key is key of element in position i of heap
  - although actually array may only hold pointers to elements

Max-Heap-Maximum

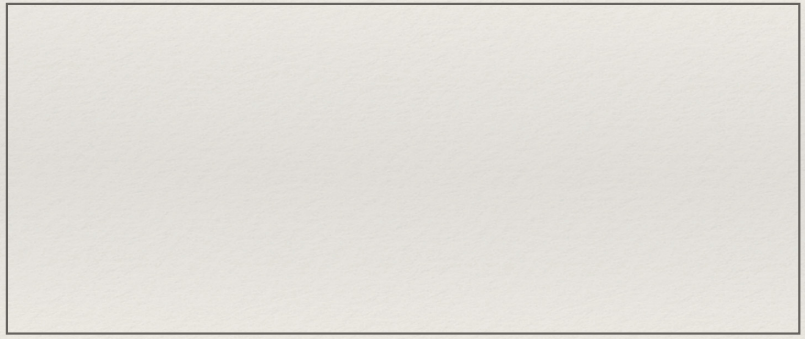
- Max-Heap-Maximum(A) // return max element, don't delete
  - if A.heapsize < 1
    - error "heap underflow"
  - return A[1]



Max-Heap-Maximum

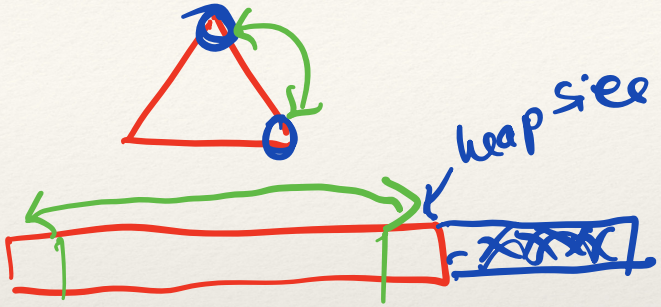
- Max-Heap-Maximum(A) // return max element, don't delete
  - if A.heapsize < 1
    - error "heap underflow"
  - return A[1]

Max-Heap-Extract-Max(A)

- Max-Heap-Extract-Max(A)
  - max = Max-Heap-Maximum(A)
  - 
  - return max

Max-Heap-Extract-Max(A)

- Max-Heap-Extract-Max(A)
  - max = Max-Heap-Maximum(A)
  - A[1] = A[A.heapsize] // move last element in heap to root
  - A.heapsize = A.heapsize-1 // update heapsize
  - Max-Heapify(A,1) // restore heap order
  - return max



Max-Heap-Extract-Max(A)

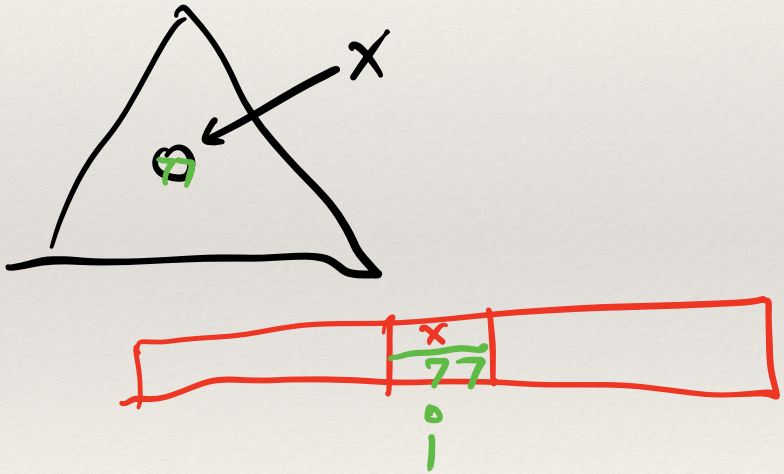
- Max-Heap-Extract-Max(A)
  - max = Max-Heap-Maximum(A)
  - A[1] = A[A.heapsize] // move last element in heap to root
  - A.heapsize = A.heapsize-1 // update heapsize
  - Max-Heapify(A,1) // restore heap order
  - return max

Worst-case Runtime? $\Theta(\log n)$

Express runtime as function of n, where n is the size of the heap.

Max-Heap-Increase-Key(A,x,k)

- Idea:
  - Locate x in heap (using mapping operation)
  - Increase key of x to k
  - Need to repair heap!



Max-Heap-Increase-Key(A,x,k)

- Idea:
  - Locate x in heap (using mapping operation)
  - Increase key of x to k
    - Potentially causes violation of max-heap property
  - "Bubble up" x by repeatedly
    - swapping it with its parent until it is less than its parent or it is the root
  - After each swap
    - Still at most one violation of max-heap property
  - Violation moves up one level after each swap (or is eliminated)

Increase-Key

pg. 176

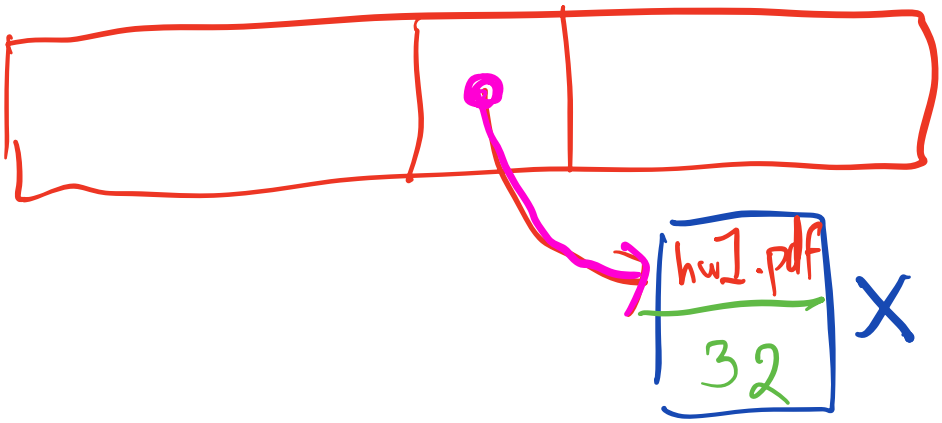
```
MAX-HEAP-INCREASE-KEY(A, x, k)
1  if k < x.key
2        error "new key is smaller than current key"
3  x.key = k
4  find the index i in array A where object x occurs
5  while i > 1 and A[PARENT(i)].key < A[i].key
6        exchange A[i] with A[PARENT(i)], updating the information that maps
             priority queue objects to array indices
7        i = PARENT(i)
```

Reset Key, Locate, Bubble Up

Runtime: $O(\log n)$ [excluding mapping operations]

Here n is number of elements in the PQ


hw1.pdf
32


hw1.pdf
32
X

Max-Heap-Increase-Key

pg. 176

```
MAX-HEAP-INCREASE-KEY(A, x, k)
1  if k < x.key
2       error "new key is smaller than current key"
3  x.key = k
4  find the index i in array A where object x occurs
5  while i > 1 and A[PARENT(i)].key < A[i].key
6       exchange A[i] with A[PARENT(i)], updating the information that maps
            priority queue objects to array indices
7       i = PARENT(i)
```

Reset Key, Locate, Bubble Up

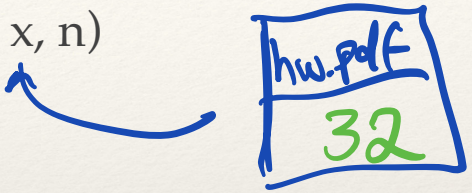
Runtime: $O(\log n)$ [excluding mapping operations]

n is number of elements in the PQ

Max-Heap-Insert(A,x,n)

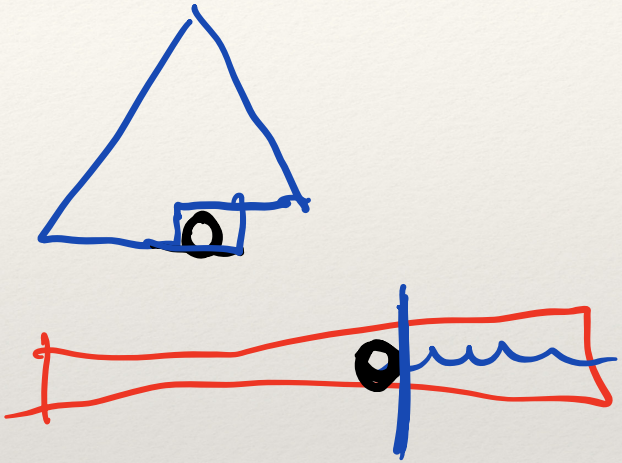


- Textbook gives pseudocode for Max-Heap-Insert(A, x, n)
  - assumes x.key already has desired key value
  - n is length of array A
  - Max-Heap implementation of PQ operation Insert
- Idea:
  - Use Max-Heap-Increase-Key(A,x,k) to help implement Max-Heap-Insert(A,x,k)



Max-Heap-Insert

```
MAX-HEAP-INSERT(A, x, n)
1  if A.heap-size == n
2        error "heap overflow"
3  A.heap-size = A.heap-size + 1
4  k = x.key
5  x.key = −∞
6  A[A.heap-size] = x
7  map x to index heap-size in the array
8  MAX-HEAP-INCREASE-KEY(A, x, k)
```



Add element after the last position in the heap, with key = $-\infty$.
Then do Max-Heap-Increase key to set its key to k.
Runtime O(log N) [excluding mapping operations]
~~n is length of array A~~
N is size of P.Q.

Max-Heap-Insert

```
MAX-HEAP-INSERT(A, x, n)
1  if A.heap-size == n
2        error "heap overflow"
3  A.heap-size = A.heap-size + 1
4  k = x.key
5  x.key = −∞
6  A[A.heap-size] = x
7  map x to index heap-size in the array
8  MAX-HEAP-INCREASE-KEY(A, x, k)
```

Add element after the last position in the heap, with key = $-\infty$.
Then do Max-Heap-Increase key to set its key to k.
Runtime O($\log N$) [excluding mapping operations]
n is length of array A, N = A.heap-size

PQ Operation Runtime: Doubly-linked list vs. Heap

Unordered

	Heap	Doubly-Linked List
Insert	$\Theta(\log n)$	$\Theta(1)$
Max	$\Theta(1)$	$\Theta(n)$
Extract-Max	$\Theta(\log n)$	$\Theta(n)$
Increase-Key*	$\Theta(\log n)$	$\Theta(1)$

w.c. runtime

Assume PQ already has n elements

[Exclude time for mapping x to its location]

*Excluding time for mapping

(locating x in data structure)

PQ Operation Runtime: Doubly-linked list vs. Heap

	Heap	Doubly-Linked List
Insert	$O(\log n)$	$\theta(1)$
Max	$\theta(1)$	$O(n)$
Extract-Max	$O(\log n)$	$O(n)$
Increase-Key	$O(\log n)$	$\theta(1)$

To compare running times, we're using n here to be the max number of elements in the PQ at any one time during the "life" of the PQ.

Sorting with a Priority Queue

- Generic-PQ-sort(A,n)
  - Insert elements of A into a PQ // Don't change array A
    // PQ is stored in a separate data struture.
  - For j = n down 1 // Put elements of priority queue into A in sorted order
    - ExtractMax(PQ) and put result in A[j]

Runtime?

Sorting with a Priority Queue

- Generic-PQ-sort(A,n)
  - Insert elements of A into a PQ // Don't change array A
    // PQ is stored in a separate data struture.
  - For j = n down 1 // Put elements of priority queue into A in sorted order
    - ExtractMax(PQ) and put result in A[j]

Runtime? Depends on how PQ is implemented.
With doubly-linked list: $\theta(n)$ total for n Insert ops
$\theta(n^2)$ total for n ExtractMax ops

wc

Total runtime? $\Theta(n^2)$

Sorting with a Priority Queue

- Generic-PQ-sort(A,n)
  - Insert elements of A into a PQ // Don't change array A
    // PQ is stored in a separate data struture.
  - For j = n down 1 // Put elements of priority queue into A in sorted order
    - ExtractMax(PQ) and put result in A[j]

Runtime? Depends on how PQ is implemented.

With heap? $O(n \log n)$ total for n Insert ops

$O(n \log n)$ total for n ExtractMax ops

Total runtime? $O(n \log n)$

Comparison to Heapsort

- Heapsort is like running the generic PQ-sorting algorithm with a heap implementation of the PQ but
  - It is "in-place". Array A used both to store the elements still in the PQ and the elements already sorted.
  - Uses Build-Heap instead of inserting elements into heap one by one
    - Takes time $\theta(n)$ to get elements into heap order, instead of $\theta(n \log n)$
- Both run in time $\theta(n \log n)$
- We say a sorting algorithm is "In-Place" if at most a constant number of elements of the input array are stored outside the input array at any time.

Comparison to Heapsort

- Heapsort is like running the generic PQ-sorting algorithm with a heap implementation of the PQ but
  - It is "in-place". Array A used both to store the elements still in the PQ and the elements already sorted.
  - Uses Build-Heap instead of inserting elements into heap one by one
    - Takes time _____ to get elements into heap order, instead of _____
- Both run in time $\theta(n \log n)$
- We say a sorting algorithm is "In-Place" if at most a constant number of elements of the input array are stored outside the input array at any time.

Some important concepts from probability

Linearity of Expectation

- We will use following important property called “Linearity of Expectation”
- For any two random variables X and Y, and constants a and b
  - $E[aX + bY] = aE[X] + bE[Y]$
- True even if X and Y are not independent!

A coin flipping problem

- Suppose flip a fair coin repeatedly, stopping as soon as get Heads, or have flipped 4 times (get 4 Tails)
- What is expected number of flips?
- Standard way to solve this:
  - Define X to be random variable which is number of flips performed (stop when get Heads)
  - $E[X]$ = Expected number of flips
    $= 1 * P[X = 1] + 2 * P[X = 2] + 3 * P[X = 3] + 4 * P[X = 4]$
    $=$

A coin flipping problem

- Suppose flip a fair coin repeatedly, stopping as soon as get Heads, or have flipped 4 times (get 4 Tails in a row)
- What is expected number of flips?
- Standard way to solve this:
  - Define X to be random variable which is total number of flips performed (if stop when get Heads or flipped 4 Tails in a row)
  - $E[X]$ = Expected number of flips

$$= 1 * P[X = 1] + 2 * P[X = 2] + 3 * P[X = 3] + 4 * P[X = 4]$$

$$= 1 * \frac{1}{2} + 2 * \frac{1}{4} + 3 * \frac{1}{8} + 4 * \frac{1}{8}$$

$$= (1/2) + (1/2) + (3/8) + (4/8) = 1\frac{7}{8}$$

A different approach to coin flipping problem

- Can solve using Indicator Random Variables
- Indicator random variable Y for an event
  - $Y = 1$ if event occurs
  - $Y = 0$ if event does not occur
- Since Y is an indicator random variable, $E[Y] = P[Y = 1]$
  - Why?

Solving coin flipping problem with indicator variables

- Again, suppose you have a fair coin
- Flip it repeatedly until either you get Heads or you've flipped it 4 times
- What is the expected number of flips you perform?
  - Let X_i be indicator variable for the event that you perform an ith flip
  - Note that the different X_i variables are not independent!

- Let X be random variable which is total number of flips performed
- $X = X_1 + X_2 + X_3 + X_4$
- Expected number of flips performed is
  - $E[X] = E[X_1 + X_2 + X_3 + X_4] =$

- Let X be random variable which is total number of flips performed
- $X = X_1 + X_2 + X_3 + X_4$
- Expected number of flips performed is
  - $$\begin{aligned} E[X] &= E[X_1 + X_2 + X_3 + X_4] \\ &= E[X_1] + E[X_2] + E[X_3] + E[X_4] \\ &= 1 + (1/2) + (1/4) + (1/8) \\ &= 1\frac{7}{8} \end{aligned}$$

- Let X be random variable which is total number of flips performed
- $X = X_1 + X_2 + X_3 + X_4$
- Expected number of flips performed is
  - $$\begin{aligned} E[X] &= E[X_1 + X_2 + X_3 + X_4] \\ &= E[X_1] + E[X_2] + E[X_3] + E[X_4] \\ &= 1 + (1/2) + (1/4) + (1/8) \\ &= 1\frac{7}{8} \end{aligned}$$

Quicksort

Quicksort

- Very practical (with tuning)
- Divide-and-conquer
- Idea:
  - Pick an element x of the array. Call it the "pivot"
  - Partition the other elements with respect to the pivot, as follows:
    Elements $\leq$ x go on left side of array (low side), elements > x go on right side of array (high side)
    Pivot x goes in between the two sides
  - Recursively sort the low side
  - Recursively sort the high side

Partition example

- 4 2 8 9 3 1 7 5
- Suppose pick 5 to be the pivot
- If use extra array, very easy to partition around pivot in time $\theta(n)$
- We will do it in-place in time $\theta(n)$ (slightly more complicated)

Partition example

- Idea to do in place:
  - Pick last element of array to be the pivot
  - Loop for j=1 to n-1, maintain invariant that the prefix A[1…j-1] of A is partitioned according to pivot
  - Keep variable i that is highest index of element <= x in partitioned prefix A[1…j-1]. (initialize to 0)
  - In body of loop, incorporate new element A[j] into the already partitioned prefix
  - if A[j] > x, just leave where A[j] in place
    if A[j] <= x, increment i and exchange A[i] with A[j]
  - At end: Move pivot to position i+1
- pseudocode on p. 184, for A[p…r]

Example:

Start with 4 2 3 8 9 1 7 5

Fast forward until start of iteration where j=6

4 2 3 8 9 1 7 5
  i   j

g. 184

```
PARTITION(A, p, r)
1  x = A[r]                              // the pivot
2  i = p − 1                             // highest index into the low side
3  for j = p to r − 1                    // process each element other than the pivot
4      if A[j] ≤ x                       // does this element belong on the low side?
5          i = i + 1                         // index of a new slot in the low side
6          exchange A[i] with A[j]           // put this element there
7  exchange A[i + 1] with A[r]           // pivot goes just to the right of the low side
8  return i + 1                          // new index of the pivot
```

- Note:
  - In mergesort — divide is trivial, combining results of recursive calls takes $\theta(n)$
  - In quicksort — divide takes work (partition), combining results of recursive calls is trivial

```
QUICKSORT(A, p, r)
1  if p < r
2       // Partition the subarray around the pivot, which ends up in A[q].
3       q = PARTITION(A, p, r)
4       QUICKSORT(A, p, q − 1)  // recursively sort the low side
5       QUICKSORT(A, q + 1, r)  // recursively sort the high side
```

Running Time?

```
QUICKSORT(A, p, r)
1  if p < r
2      // Partition the subarray around the pivot, which ends up in A[q].
3      q = PARTITION(A, p, r)
4      QUICKSORT(A, p, q − 1)  // recursively sort the low side
5      QUICKSORT(A, q + 1, r)  // recursively sort the high side
```

Running Time? Varies depending on how the pivots split the array